

Transformation et manipulation des données



Dr Khadidja Henni

Séance 5: Prétraitement (3)

Plan de la séance

1. Encodage one-hot (One-Hot Encoding)
2. Regroupement (discrétisation d'une variable numérique)
3. Séparation de caractéristiques (Feature Split)
4. Extraction de caractéristiques temporelles
5. Pipeline de transformation
6. Validation croisée
7. Laboratoire formatif

Prétraitement de données

Les techniques de prétraitement des données

- Liste des techniques de préparation des caractéristiques:
 1. Imputation (Complétion de données manquantes)
 2. Gestion des valeurs aberrantes (Outliers)
 3. Transformation logarithmique (Log Transform)
 4. Mise à l'échelle (Scaling)
 5. Encodage one-hot (One-Hot Encoding)
 6. Binning (Discrétisation)
 7. Opérations de regroupement (Grouping Operations)
 8. Séparation des caractéristiques (Feature Split)
 9. Extraction de dates (Extracting Date)

Encodage des variables catégorielles

Encodage

Pourquoi encoder ?

- Les algorithmes d'apprentissage automatique (comme la régression, les SVM, les réseaux de neurones, etc.) ne peuvent pas directement manipuler des valeurs **catégorielles** (ex. : pays, ville, couleur, profession).
- Il est donc essentiel de **convertir ces valeurs en nombres**, tout en **préservant leur signification sémantique**.
- Un mauvais encodage peut introduire du **biais**, de la **confusion**, ou dégrader les performances d'un modèle. Un bon encodage améliore la **précision**, la **compréhension du modèle** et sa **généralisation**.

Types de variables catégorielles

Type	Définition	Exemple
Nominale	Catégories sans ordre logique	Couleur (rouge, bleu, vert)
Ordinale	Catégories avec ordre logique	Taille (petit, moyen, grand)
Binaire	Deux modalités	Sexe (H/F), Oui/Non

Techniques d'encodage

1. Label Encoding (LabelEncoder)

- Associe un entier unique à chaque modalité.
- **Risque de biais** pour les données **nominales** car l'algorithme peut interpréter une hiérarchie artificielle (**Vert > Rouge > Bleu** Et peut-être même que la **distance entre Vert et Bleu = 2** a du sens (alors que non)).
- **Adapté aux variables ordinales** (avec ordre implicite ou explicite).

Taille	Encodée
petit	0
moyen	1
grand	2

```
from sklearn.preprocessing import LabelEncoder  
le = LabelEncoder()  
data['taille'] = le.fit_transform(data['taille'])
```

Techniques d'encodage

2. One-Hot Encoding (pd.get_dummies ou OneHotEncoder)

- Crée une colonne binaire pour chaque catégorie.
- **Aucune perte d'information.**
- Peut provoquer une **explosion du nombre de colonnes** si beaucoup de modalités.

Avantages :

- Interprétable : chaque colonne = une modalité
- Évite le biais d'ordre
- Compatible avec la plupart des modèles
- **Inconvénients :**
- Non scalable si > 100 modalités (ex. : pays, métiers...)

```
import pandas as pd  
pd.get_dummies(data['ville'])
```

```
pd.get_dummies(data['ville'], drop_first=True)
```

User	Ville
1	Roma
2	Madrid
1	Madrid
3	Istanbul
2	Istanbul
1	Istanbul
1	Roma

User	Istanbul	Madrid
1	0	0
2	0	1
1	0	1
3	1	0
2	1	0
1	1	0
1	0	0

- Roma est codée implicitement : Istanbul = 0 et Madrid = 0
- Madrid est codée : Madrid = 1
- Istanbul est codée : Istanbul = 1

Techniques d'encodage

3. Ordinal Encoding (OrdinalEncoder) Comme le LabelEncoder, mais permet de **définir un ordre manuel** entre les modalités.

- Une variable Taille avec des modalités : ['Petit', 'Moyen', 'Grand']

Cet ordre est **logique** et **significatif** : Grand > Moyen > Petit.

- L'OrdinalEncoder permet de **préserver cet ordre** : $\rightarrow$ ['Petit', 'Moyen', 'Grand'] $\rightarrow$ [0, 1, 2]

```
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder(categories=[['faible', 'moyen', 'élevé']])
oe.fit_transform(data[['niveau']])
```

- Certains modèles (ex. : régression, arbre, etc.) peuvent **exploiter cette relation d'ordre** :
- Grand implique plus d'intensité que Moyen, etc.
- Permet aussi d'éviter la création de multiples colonnes (comme en One-Hot)

Ne jamais encoder une variable comme ordinale si l'ordre n'a aucun sens.

Encodage

Target Encoding: Il consiste à remplacer chaque catégorie par la **moyenne de la variable cible** pour cette catégorie.

Exemple:

Si la variable Profession contient :

On remplace chaque modalité par ce salaire moyen.

Profession	Salaire moyen
Professeur	50 000
Ingénieur	70 000
Artiste	30 000

Pourquoi ne jamais le faire avant le split train/test ?

- Parce que vous utilisez **des informations issues de la variable cible du jeu complet, y compris le test.**
- Cela signifie que vous **injectez dans l'entraînement des informations** que le modèle **n'est pas censé connaître à l'avance.**

Conséquence : Le modèle a une **connaissance artificielle du test**, donc :

- Les performances seront **surgonflées en validation**
- Mais **mauvaises en production** → data leakage.

Toujours faire le target encoding après avoir séparé le train/test et appliquer l'encoding calculé uniquement sur les données d'entraînement.

Encodage

Binary Encoding (via category_encoders):

Quand vous avez une **colonne avec beaucoup de catégories distinctes** (ex. : codes postaux, produits, utilisateurs), le **One-Hot Encoding devient inutilisable** : trop de colonnes, mémoire saturée.

Principe du Binary Encoding :

1. On attribue un entier à chaque catégorie (comme un LabelEncoder)
2. On encode cet entier en **binaire**
3. Chaque chiffre binaire devient une colonne

Exemple : Pour **8 catégories**, on n'a besoin que de **3 colonnes** ($2^n \geq 8 \rightarrow n = 3$)

Avantages :

- Beaucoup plus **compact** que le One-Hot
- Pratique pour les variables avec **centaines ou milliers** de valeurs

Limites :

- Moins intuitif/interprétable
- Parfois un peu **bruyant** (car l'ordre est imposé artificiellement)

Encodage

Catégorie	ID	Binaire (sur 3 bits)	Résultat
A	1	001	[0, 0, 1]
B	2	010	[0, 1, 0]
C	3	011	[0, 1, 1]
D	4	100	[1, 0, 0]

?

```
import pandas as pd
import category_encoders as ce

# Données de départ
df = pd.DataFrame({
    'fruit': ['pomme', 'banane', 'orange', 'kiwi',
             'banane', 'pomme', 'raisin']})

# Étape 1 : Label Encoding (pour récupérer l'ID entier utilisé)
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['fruit_id'] = le.fit_transform(df['fruit'])

# Étape 2 : Encodage binaire
binary_encoder = ce.BinaryEncoder(cols=['fruit'])
df_binary = binary_encoder.fit_transform(df[['fruit']])

# Étape 3 : Combine original + ID + binaire
result = pd.concat([df['fruit'], df['fruit_id'], df_binary], axis=1)

# Affichage propre
print("Tableau fruit + ID + encodage binaire :\n")
print(result)
```

Encodage

Quelle méthode choisir ?

Situation	Méthode recommandée
Variable nominale à faible cardinalité	One-Hot Encoding
Variable nominale à forte cardinalité	Target / Binary Encoding
Variable ordinale avec ordre explicite	Ordinal Encoding
Variable binaire	Mapping booléen (0/1)
Données très déséquilibrées	Groupeement + TargetEnc

Bonnes pratiques

- Toujours encoder **après avoir split les données** (train/test)
- Enregistrer les objets fit() (LabelEncoder, OneHotEncoder) pour réutilisation sur les nouvelles données
- Ne jamais encoder les données de test séparément (risque de nouvelles colonnes ou valeurs non vues)
- Intégrer l'encodage dans un **pipeline** avec ColumnTransformer

Regroupement (discrétisation d'une variable numérique)

Regroupement (binning)

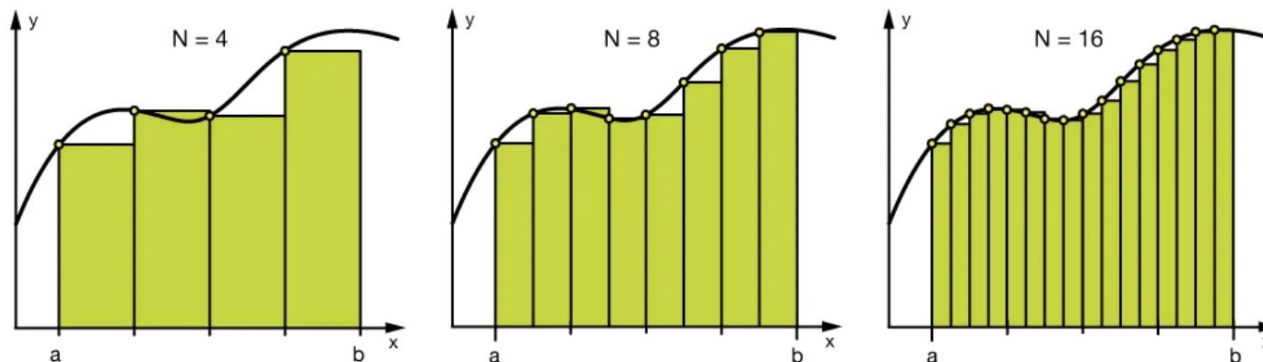
Pourquoi faire du regroupement (binning) ?

Le **regroupement (binning)** est une technique qui consiste à **regrouper des valeurs proches** ou **des catégories rares** pour :

- **réduire la complexité du modèle**
- **éviter le surajustement (overfitting)**
- **stabiliser les prédictions** quand certaines modalités sont trop spécifiques ou sous-représentées
- Cependant, **ce regroupement entraîne une perte d'information** (on simplifie le signal).

Le binning est donc un compromis :

- Trop de détails → risque de surajustement
- Trop de regroupements → perte de précision



#Numerical Binning Example

Value	Bin
0-30	-> Low
31-70	-> Mid
71-100	-> High

#Categorical Binning Example

Value	Bin
Spain	-> Europe
Italy	-> Europe
Chile	-> South America
Brazil	-> South America

Regroupement (binning)

Exemple : Binning numérique (âge)

- Imaginons un modèle de classification de clients selon le risque de crédit.
- Tu disposes d'une variable continue âge allant de 18 à 80 ans.
- Si tu la laisses telle quelle :
- Un arbre de décision pourra faire des splits très fins (ex. : $\text{âge} < 26.5$, $\text{âge} \geq 63.7$)
- Cela crée un **modèle trop spécifique** qui ne généralise pas bien

```
import pandas as pd

data = pd.DataFrame({
    'age': [22, 25, 31, 35, 45, 52, 60, 68, 73, 78]
})

bins = [18, 30, 50, 80]
labels = ['Jeune', 'Adulte', 'Senior']

data['age_group'] = pd.cut(data['age'], bins=bins, labels=labels)

print(data)
```

	age	age_group
0	22	Jeune
1	25	Jeune
2	31	Adulte
3	35	Adulte
4	45	Adulte
5	52	Senior
6	60	Senior
7	68	Senior
8	73	Senior
9	78	Senior

Le modèle travaillera maintenant avec 3 modalités (Jeune, Adulte, Senior) au lieu de 63 valeurs possibles, ce qui rend le modèle :

- plus lisible
- moins sensible aux petites variations
- moins sujet au surajustement

Regroupement (binning)

#Numerical Binning Example

```
data['bin'] = pd.cut(data['value'], bins=[0,30,70,100], labels=
["Low", "Mid", "High"])
```

	value	bin
0	2	Low
1	45	Mid
2	7	Low
3	85	High
4	28	Low

#Categorical Binning Example

	Country
0	Spain
1	Chile
2	Australia
3	Italy
4	Brazil

```
conditions = [
    data['Country'].str.contains('Spain'),
    data['Country'].str.contains('Italy'),
    data['Country'].str.contains('Chile'),
    data['Country'].str.contains('Brazil')]
```

```
choices = ['Europe', 'Europe', 'South America', 'South America']
```

```
data['Continent'] = np.select(conditions, choices, default='Other')
```

	Country	Continent
0	Spain	Europe
1	Chile	South America
2	Australia	Other
3	Italy	Europe
4	Brazil	South America

Séparation de caractéristiques (Feature Split)

Feature Split

Objectif

De nombreuses colonnes dans les jeux de données réels peuvent contenir **plusieurs informations imbriquées dans un seul champ**.

Par exemple :

- Une colonne Nom complet → contient **Prénom + Nom**
- Une colonne Adresse → contient **Numéro, Rue, Ville, Code postal**
- Une colonne Date → contient **Jour, Mois, Année**

Séparer ces sous-composantes peut permettre de :

- **simplifier l'analyse,**
- **extraire des variables pertinentes** pour la modélisation,
- **améliorer l'interprétabilité** des modèles.

Colonne source	Ce qu'on peut extraire
nom_complet	prénom, nom
date_achat	année, mois, jour, jour de la semaine
lieu	ville, pays
adresse complète	code postal, ville
produit	catégorie, sous-catégorie, code article

Feature Split

1. str.split() — découpe simple par séparateur

```
df[['prenom', 'nom']] = df['nom_complet'].str.split(' ', n=1, expand=True)
```

- n=1 : nombre maximum de séparations
- expand=True : retourne un DataFrame directement

Exemple :

"Khadidja Henni" → "Khadidja" et "Henni »

2. str.extract() — extraction par motif avec regex

Permet d'extraire des éléments plus complexes, comme un **code**, une **date**, une **valeur dans une chaîne**.

```
df[['ville', 'pays']] = df['lieu'].str.extract(r'(\w+),\s*(\w+)')
```

Exemple :

'Québec, Québec' → 'Paris' et 'Canada'

Feature Split

3. Regex avec plusieurs groupes

```
df[['num_rue', 'rue']] = df['adresse'].str.extract(r'(\d+)\s+(.*)')
```

- `(\d+)` : nombre (numéro de rue)
- `(.*)` : tout le reste (nom de rue)

Exemple : "14 rue Victor Hugo" → 14, rue Victor Hugo

4. `str.contains()`, `str.replace()` — nettoyage de chaînes

- Supprimer les accents, espaces en trop, parenthèses, etc.
- Nettoyage avant séparation :

```
df['nom'] = df['nom'].str.strip().str.lower().str.replace(r'[^a-z ]', '', regex=True)
```

Feature Split

Pourquoi cette étape est importante ?

- De nombreux **modèles supervisés** ou **algorithmes d'analyse** ne peuvent pas exploiter directement une chaîne complexe.
- La **séparation permet de transformer des données semi-structurées en variables exploitables**.
- Elle améliore la qualité de la **visualisation**, du **regroupement**, et de l'**encodage**.

Bonnes pratiques

- Toujours inspecter visuellement les motifs avant d'appliquer split ou extract
- Nettoyer les valeurs avant transformation (strip, lower, etc.)
- Ajouter une vérification post-extraction (ex. : `df.isnull().sum()`) pour repérer les lignes mal formatées)
- Documenter les choix de séparation (ex. : `split(' - ')`, `extract(r'...')`, etc.)

Extraction de caractéristiques temporelles

Extraction de caractéristiques temporelles

Objectif

Les colonnes de type datetime (date et heure) contiennent souvent **plusieurs dimensions temporelles implicites** :

- **année, mois, jour, heure, jour de la semaine, trimestre, période de la journée**, etc.

Ces sous-caractéristiques peuvent révéler des **motifs temporels** utiles pour :

- les modèles prédictifs,
- les analyses saisonnières,
- les visualisations temporelles.

Pourquoi extraire des composantes temporelles ?

Besoin métier	Variable extraite
Effet saisonnier sur les ventes	mois, trimestre
Effet du week-end sur la fréquentation	jour de la semaine
Identification d'heures creuses	heure, période (matin, soir...)
Évolution sur plusieurs années	année
Analyse hebdomadaire	semaine de l'année

Extraction de caractéristiques temporelles

1. Convertir la colonne en datetime

```
df['date_achat'] = pd.to_datetime(df['date_achat'])
```

2. Extraire les composantes

```
df['année'] = df['date_achat'].dt.year
df['mois'] = df['date_achat'].dt.month
df['jour'] = df['date_achat'].dt.day
df['jour_semaine'] = df['date_achat'].dt.dayofweek # 0 = lundi, 6 = dimanche
df['nom_jour'] = df['date_achat'].dt.day_name()      # lundi, mardi, etc.
df['heure'] = df['date_achat'].dt.hour
df['trimestre'] = df['date_achat'].dt.quarter
```

Exemple:

date_achat	année	mois	jour	jour_semaine	heure
2023-07-14 10:15	2023	7	14	4 (vendredi)	10
2023-07-15 18:50	2023	7	15	5 (samedi)	18

```
df['saison'] = pd.cut(df['mois'],
                      bins=[0, 3, 6, 9, 12],
                      labels=['Hiver', 'Printemps', 'Été', 'Automne'])
```

Extraction de caractéristiques temporelles

Utilité dans les modèles

- **Classification prédictive** : prédire le taux de churn selon le mois
- **Régression de séries temporelles** : ajout de mois, jour_semaine, fête comme variables exogènes
- **Segmentation de clientèle** : comportement horaire ou hebdomadaire
- **Optimisation logistique** : analyse des pics d'activité (par jour ou saison)

Bonnes pratiques

- Toujours convertir les colonnes en datetime **dès l'importation**
- Ne pas oublier le format (dayfirst=True, etc.) si ambigu
- Visualiser la distribution temporelle (df['mois'].value_counts()) pour détecter des biais

Pipelines et ColumnTransformer

Pipeline et ColumnTransformer

Pourquoi utiliser un pipeline ?

Un **pipeline** permet de **chaîner automatiquement toutes les étapes de traitement des données**, de manière propre, cohérente et reproductible.

C'est un outil fondamental pour tout data scientist ou ingénieur IA.

Avantages principaux :

- Exécution **structurée** et **ordonnée** des transformations
- **Moins d'erreurs** : pas besoin de répéter les mêmes fit/transform manuellement
- Facilité d'**intégration dans un modèle de production**
- **Réutilisable** et **sauvegardable** facilement
- Compatible avec GridSearchCV, cross_val_score, etc.

Pipeline et ColumnTransformer

Étapes classiques dans un pipeline

- **Imputation** des valeurs manquantes (SimpleImputer)
- **Transformation** des variables numériques (scaling, log, binning...)
- **Encodage** des variables catégorielles (sauf targetencoding)
- **Modélisation** (régression, arbre, etc.)

Toutes ces étapes sont encapsulées dans un seul objet pipeline.

ColumnTransformer pour les données mixtes

- Quand un jeu de données contient à la fois :
- des **variables numériques** (âge, revenu...)
- des **variables catégorielles** (sexe, profession...)

Il faut appliquer des **transformations différentes** selon le type de variable.
C'est exactement ce que permet le ColumnTransformer.

Pipeline et ColumnTransformer

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

# Colonnes du dataset
num_features = ['age', 'salaire']
cat_features = ['sexe', 'region']

# Pipeline pour les numériques : imputation + standardisation
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Pipeline pour les catégorielles : imputation + one-hot
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

# Transformation combinée
preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_features),
    ('cat', cat_pipeline, cat_features)
])

# Pipeline complet
pipeline = Pipeline([
    ('pretraitement', preprocessor),
    ('modele', LogisticRegression())
])
```

```
pipeline.fit(X_train, y_train)
preds = pipeline.predict(X_test)
```

```
import joblib

# Sauvegarder
joblib.dump(pipeline, 'mon_pipeline.joblib')

# Recharger
pipeline_recharge = joblib.load('mon_pipeline.joblib')

# Réutiliser
pipeline_recharge.predict(nouvelles_donnees)
```

Pipeline et ColumnTransformer

Bonnes pratiques

- Toujours séparer les données train / test **avant de fitter le pipeline**
- Ne jamais fitter un scaler ou encodeur sur l'ensemble du jeu de données
- Bien nommer chaque étape du pipeline ('scaler', 'encoder', etc.)
- Utiliser `handle_unknown='ignore'` avec OneHotEncoder pour les nouvelles catégories
- Intégrer tout dans un seul pipeline, **même le modèle final**

Validation croisée

Validation croisée

Pourquoi valider ?

- Un modèle peut **apprendre par cœur** les données d'entraînement.
- Il faut mesurer sa capacité à **généraliser** à des données **nouvelles**.
- Séparer le jeu en :
 - **train** : pour l'apprentissage
 - **test/val** : pour l'évaluation

Une seule division **train/test** peut être **biaisée** selon la façon dont les données sont réparties.

Principe de la cross-validation

- On **répète** le processus d'entraînement/évaluation **plusieurs fois** sur **des sous-échantillons différents**.
- Chaque observation est utilisée **une fois comme test**, le reste comme apprentissage.

Validation croisée

Exemple : K-Fold

K-Fold Cross Validation :

- On divise les données en **K blocs** (ou folds) de taille égale.
- À chaque tour :
 - Un fold = test
 - Les K-1 autres = entraînement
- On répète K fois.

Fold	Test	Entraînement
1	Fold 1	Folds 2,3,4,5
2	Fold 2	Folds 1,3,4,5
3	Fold 3	Folds 1,2,4,5
...	...	...

Exemple avec $K = 5$:

On moyenne les scores des 5 modèles.

Validation croisée

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris

# Chargement des données
X, y = load_iris(return_X_y=True)

# Modèle
model = LogisticRegression(max_iter=200)

# Validation croisée (5 folds)
scores = cross_val_score(model, X, y, cv=10)

# Affichage
print("Scores par fold :", scores)
print(f"Moyenne des scores : {scores.mean():.3f}")
print(f"Écart-type des scores : {scores.std():.3f}")
```

```
Scores par fold : [1.          0.93333333 1.          1.          0.93333333 0.93333333
 0.93333333 1.          1.          1.          ]
Moyenne des scores : 0.973
Écart-type des scores : 0.033
```

Validation croisée

Variantes de K-Fold

Type	Description
StratifiedKFold	Divise les données en K sous-ensembles en préservant la proportion des classes dans chaque fold.
Leave-One-Out (LOO)	À chaque itération, une seule observation est testée, les autres sont utilisées pour l'apprentissage. (par exemple données biomédicales patient par patient)
GroupKFold	Garantit que toutes les données d'un même groupe restent ensemble (dans train ou test, jamais les deux).
RepeatedKFold	Répète K-Fold avec des découpages aléatoires différents pour une évaluation plus robuste.

Bonnes pratiques

- Choisir StratifiedKFold pour des classes déséquilibrées
- Utiliser shuffle=True pour éviter des partitions biaisées
- Intégrer la cross-validation **dans le pipeline**
- Toujours tester les hyperparamètres avec **validation croisée** (pas sur test direct !)

Laboratoire formatif

Laboratoire formatif

Prétraitement & Modélisation: Appliquer les techniques de préparation de données et de validation croisée sur un jeu de données clients.

Partie 1 — Prétraitement

1. Séparer les colonnes ``nom_complet`` en ``prenom`` et ``nom``.
2. Extraire la ville depuis la colonne ``adresse``.
3. Créer une variable ``age_groupe`` (Jeune, Adulte, Senior) avec ``pd.cut()``.
4. Discrétiser ``revenu_mensuel`` en quartiles (``pd.qcut``).
5. Extraire depuis ``date_achat`` : l'année, le mois, le jour de la semaine.
6. Regrouper les professions peu fréquentes en catégorie ``autre``.
7. Encoder :
 - ``sexe`` en One-Hot
 - ``profession`` en Ordinal ou One-Hot
 - ``produit`` en BinaryEncoding
8. Standardiser les colonnes ``age`` et ``revenu_mensuel`` dans un pipeline.

Laboratoire formatif

Prétraitement & Modélisation: Appliquer les techniques de préparation de données et de validation croisée sur un jeu de données clients.

Partie 2 — Modélisation & Pipeline

1. Créer un ``ColumnTransformer`` pour appliquer les bons traitements :
 - numériques : imputation + standardisation
 - catégorielles : imputation + encodage
2. Intégrer ce ``ColumnTransformer`` dans un pipeline avec :
 - ``KNeighborsClassifier``
 - ``DecisionTreeClassifier``
3. Comparer les performances avec « validation croisée (K=5) » :
 - Avec « pipeline brut » (sans transformation)
 - Avec « pipeline complet » (prétraitement + modèle)
4. Afficher pour chaque modèle :
 - les « scores par fold »
 - la « moyenne et l'écart-type